

add.py 2.0

Trimačiai modeliai, sukurti vien tik programiniu kodu – Python arba C++



Kas naujo 2.0 versijoje

Loginės operacijos

union, intersect, difference – sąjunga, sankirta ir skirtumas. Parašyta nuo nulio, be jokių bibliotekų.

Taisyklingieji briaunainiai ir geodezinė sfera

tetrahedron ... icosahedron, sucentruoti; sphere()
dabar – kupolas iš trikampių.

Glotnūs paviršiai

catmull_clark ir smooth(): apibendrintas Catmull–Clark algoritmas, bet koks langelių skaičius ant briaunos.

Viršūnių įrankiai

set_vertex, neighbors, valence, dual, truncate – futbolo kamuolys iš ikosaedro viršūnių.

Vardiniai paviršiai, spalvų funkcijos, detalės

25 paviršių pagal vardą; color=lambda u, v: ...; beam, wheel, gear, roof, tree, text ...

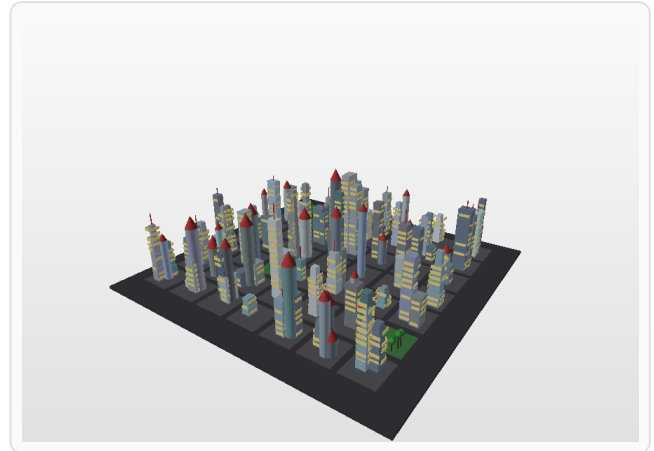
Išsaugojimas Sketchfab

check() prižiūri 50 MB / 50 spalvų ribas; stiklas (transparent) ir tekstūros įrašomi į .mtl.

Viskas, kas parašyta senajai 1.2 versijai, veikia be jokių pataisymų – o visa tai dabar yra ir C++ kalbai: add.hpp, vienas antraštės failas, tos pačios funkcijos, tie patys modeliai.

Kūrybinė užduotis

- Sukurti 3D modelį naudojant tik pirminį programos tekstą.
- Modelio failo formatas – OFF arba OBJ.
- Bent 10 000 daugiakampių.
- Bent 3 skirtingos spalvos.
- Algoritme privalo būti for arba while ciklas.
- Bent 1 parametras, nuo kurio priklauso modelio forma.
- Negalima naudoti 3D modeliavimui skirtų programų.
- Modelį galima kurti individualiai arba komandoje dviese.



Vienas skaičius (SEED) nulemia visą miestą

add.check()

pasako, kurie reikalavimai jau įvykdyti.

Rekomenduojami įrankiai

Python 3

Daugiau nieko diegti nereikia. add.py naudoja tik math ir random.

add.py

Vienas failas šalia jūsų programos. Parsisiunčiamas iš GitHub (projektas add3d).

tools/preview.py

Modelio peržiūra be jokios kitos programos: sukuria PNG paveikslėlį.

GeoGebra

Patogu patikrinti kreivių ir paviršių formules prieš rašant kodą.

MeshLab

OFF peržiūra ir konvertavimas. Nebebūtinas – OBJ išsaugo pats add.py.

Sketchfab.com

.obj + .mtl viename archyve – ir modelį galima pasukioti naršyklėje.

Modelis – tai du sąrašai

```
OFF
8 6 0
-1 -1 -1
-1 -1 1
...
4 0 4 5 1 255 0 0
4 0 1 3 2 255 0 0
...
```

OFF – formato vardas

8 6 0 – viršūnių, sienų ir briaunų skaičius

toliau – viršūnių koordinatės

4 0 4 5 1 – keturkampis, jungiantis viršūnes 0, 4, 5 ir 1

255 0 0 – sienos spalva

Visos šios bibliotekos braižymo funkcijos daro tą patį: apskaičiuoja taškus ir prideda sienas.

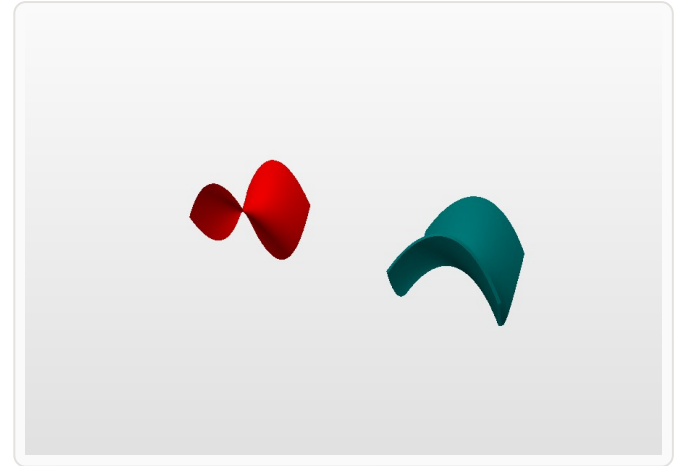
```
import add

add.box([0, 0, 0], 2, "red")
add.save("kubas.off")
```

Tas pats kubas – keturios eilutės.

Vidinė ir išorinė siena

- Žvelgiant į modelį iš išorės, viršūnių indeksus reikia išdėstyti prieš laikrodžio rodyklę.
- Žvelgiant iš vidaus – pagal laikrodžio rodyklę.
- Apversta siena atvaizduojama juoda arba visai nerodoma.
- Bibliotekos figūros orientuojamos pačios – galvoti apie tai reikia tik rašant `add.polygon(...)` ranka.
- **`add.fix_normals(M)` sutvarko visą modelį; `add.check()` pasako, ar paviršius uždaras.**



Tas pats balnas: lakštas, dvipusis ir tūrinis

Scena, sluoksniai ir modeliai

```
add.box([0, 0, 0], 1, "red")
plyta = add.layer()      # scena vėl tuščia

for i in range(10):
    add.mesh(add.move(plyta, [i * 1.2, 0, 0]))

add.save("siena.off")
```

**Sukurk vieną kartą –
padėk daug kartų.**

Transformacijos niekada nekeičia joms
perduoto modelio – jos grąžina naują.

layer()

Paima sceną kaip modelį ir palieka sceną tuščią.

mesh(M)

Grąžina modelį atgal į sceną.

merge([...])

Sujungia kelis modelius į vieną.

push() / pop()

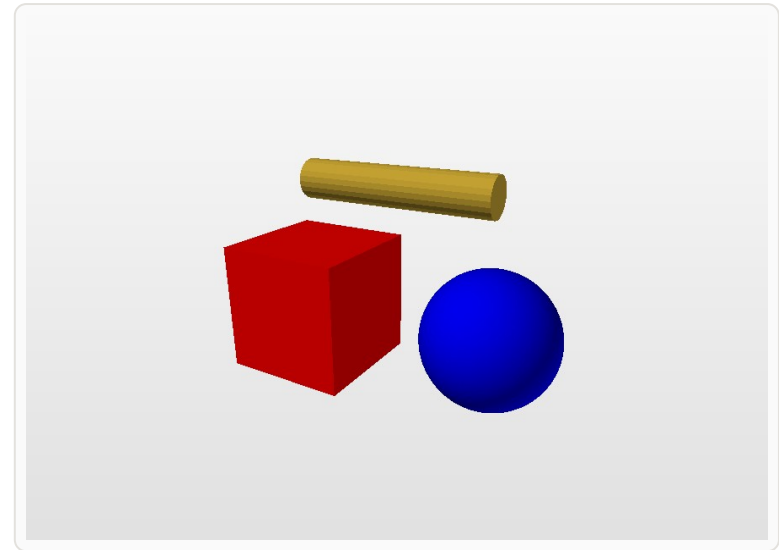
Funkcijoje, kuriančioje atskirą detalę: padeda sceną į
šalį ir po to grąžina.

Pirmasis modelis

```
import add

add.box([0, 0, 0], 2, "red")
add.sphere([3, 0, 0], 1, 20, "blue")
add.cylinder([0, 2, 0], [3, 2, 0],
            0.3, 24, "gold")

add.check()
add.save("pirmas.off")
```



vertices	2460	
!! polygons	2478	(need 10000)
OK colours	3	(need 3)
OK closed surface	yes	

check() ataskaita: dar trūksta daugiakampių.

Užtenka import add

```
import add

for i in range(12):
    a = 2 * add.pi * i / 12
    p = [3 * add.cos(a), 0, 3 * add.sin(a)]
    add.sphere(p, 0.4, 8, add.hsv(i / 12))

add.seed(7)
add.box([add.uniform(-3, 3), 0, 0], 1, "red")
```

add.sin, add.pi, add.sqrt ...

Viskas iš math – tiesiai iš add.

add.randint, add.seed,

add.Random

Ir viskas iš random.

Modelio faile – lygiai vienas importas. (Įprastas import math irgi veikia.)

Visi examples/ pavyzdžiai parašyti taip: import add ir daugiau nieko.

C++

Tas pats modelis C++ kalba

Python

```
import add

add.box([0, 0, 0], 2, "red")
add.sphere([3, 0, 0], 1, 20, "blue")
add.cylinder([0, 2, 0], [3, 2, 0],
            0.3, 24, "gold")

add.save("pirmas.off")
```

C++

```
#include "add.hpp"

int main() {
    add::box({0, 0, 0}, 2, "red");
    add::sphere({3, 0, 0}, 1, 20, "blue");
    add::cylinder({0, 2, 0}, {3, 2, 0},
                 0.3, 24, "gold");
    add::save("pirmas.off");
}
```

Baitas į baitą tas pats failas. add.hpp – visa biblioteka viename antraštės faile, naudojanti tik standartinę C++17 biblioteką: kiekviena add.py funkcija tuo pačiu vardu, su tais pačiais parametrais ir numatytosiomis reikšmėmis.

```
g++ -std=c++17 -O2 pirmas.cpp -o pirmas && ./pirmas
```

C++

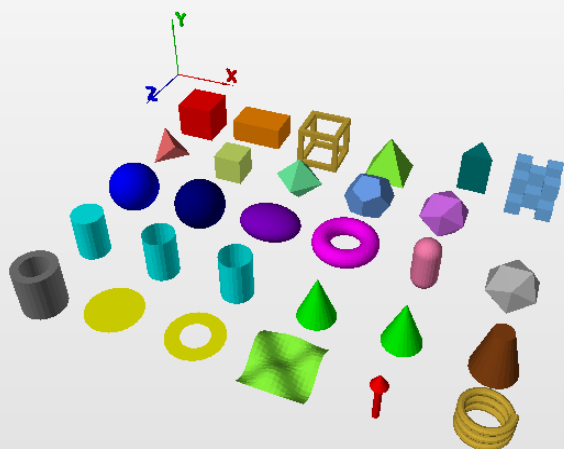
Iš Python į C++

Python	C++
<code>import add</code>	<code>#include "add.hpp"</code>
<code>[1, 2, 3]</code>	<code>{1, 2, 3}</code>
<code>add.bend(M, 0.4, around=2)</code>	<code>add::bend(M, 0.4, 1, 2)</code>
<code>add.bbox()</code>	<code>add::bbox()</code>
<code>lambda t: [t, 0, 0]</code>	<code>[](double t) { return add::Point{t, 0, 0}; }</code>
<code>add.make(add.box, [0, 0, 0], 2)</code>	<code>add::make([] { add::box({0, 0, 0}, 2); })</code>
<code>M.V, M.F, M.C</code>	<code>M.V, M.F, M.C</code>
<code>add.union(a, b)</code>	<code>add::union_(a, b)</code>
<code>print(add.center(M))</code>	<code>std::cout << add::center(M);</code>

Vardinių argumentų nėra: jie duodami iš eilės (bend: kampas, ašis, around). Praleistas tinklas reiškia sceną, kaip ir Python.
union – C++ bazinis žodis, todėl union_. Taškai ir spalvos spausdinami taip pat, kaip Python.

Patikrinta kiekvienai funkcijai, visiems 235 dokumentacijos pavyzdžiams ir 43 pavyzdinėms programoms, su GCC ir Clang. ARM kompiuteryje (Mac su Apple Silicon) kompiliuokite ir su -ffp-contract=off.

Paruoštos figūros



Kūnai

box, cuboid, rounded_box, frame,
pyramid, prism, polyhedron, voxels

Apvalūs

sphere, hemisphere, uvsphere,
ellipsoid, torus, capsule, cylinder,
tube, cup, cone, cone_open, frustum,
pipe

Plokšti

polygon, triangle, quad, disc, ring,
grid

Detalės

beam, arch, stairs, gear, wheel, roof,
column, bricks, tree, pixels,
heightmap, text

examples/02_primitives.py – visos figūros vienoje programoje.

parametric() – pagrindinė funkcija

```
def toras(u, v):  
    return [(5 + add.cos(u)) * add.cos(v),  
            add.sin(u),  
            (5 + add.cos(u)) * add.sin(v)]  
  
add.parametric(toras, 0, 2*add.pi, 40,  
               0, 2*add.pi, 80,  
               "gold", wrap_u=True, wrap_v=True)
```



wrap_u, wrap_v

Uždaro paviršių be siūlės ir be pasikartojančių viršūnių.

thickness=

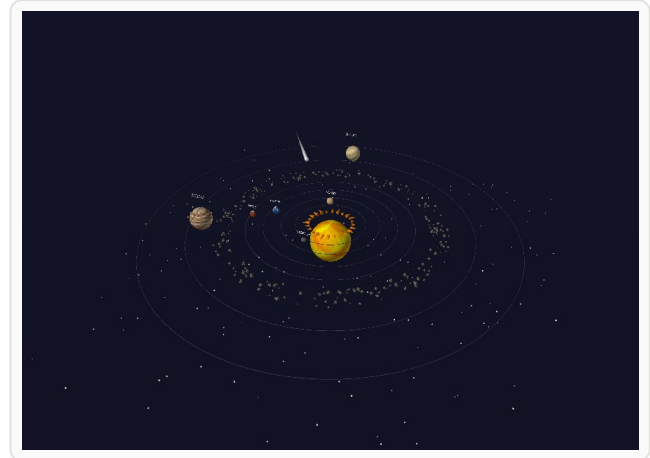
Suteikia lakštui tikrą storį – gaunamas uždaras kūnas.

color=lambda u, v: ...

Nuspalvina kiekvieną langelį pagal jo parametrus.

Spalvų funkcijos

```
add.parametric(f, 0, 1, 40, 0, 1, 40,  
              color=lambda u, v: add.hsv(u))  
  
add.revolve(vaza, A, B, 0, 4, 60, 40,  
           color=lambda t, a: "red" if int(t) % 2 else "white")  
  
add.curve(spirale, 0, 20, 300, 12, 0.2,  
         color=lambda t, a: add.hsv(t / 20))
```



Planetos: sukiniai, nuspalvinti pagal (platumą, ilgumą)

Funkcija kviečiama kiekvienam langeliui su dviem paviršiaus parametrais: (u, v) – parametric, (t, kampas) – revolve ir curve, (x, z) – grid. Juostos, šachmatų lentos, žemėlapiai ir perėjimai kainuoja vieną lambda.

Jau turimam modeliui: color_by(M, lambda p: ...) spalvina pagal padėtį.

Viena lygtis, dvylika formų

Gielio superformulė (2003)

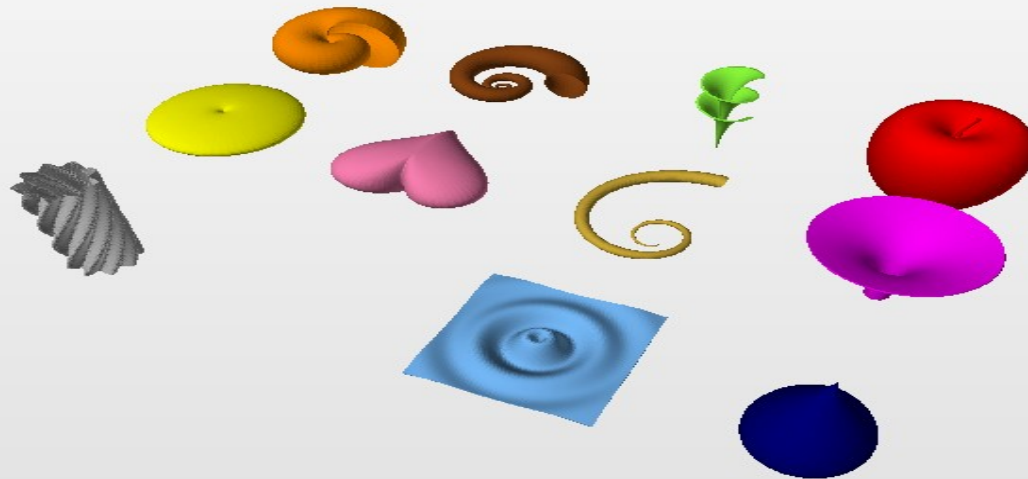
$$r(t) = (|\cos(m \cdot t/4)/a|^{\frac{1}{n_2}} + |\sin(m \cdot t/4)/b|^{\frac{1}{n_3}})^{\frac{1}{n_1}}$$

Šeši skaičiai – ir gaunama jūrų žvaigždė, gėlė, kristalas arba paprasčiausias rutulys.

Būtent to ir prašo užduotis: vienas parametras, keičiantis formą.

`examples/08_surfaces_superformula.py`

Paviršiai, kurie panašūs į daiktus

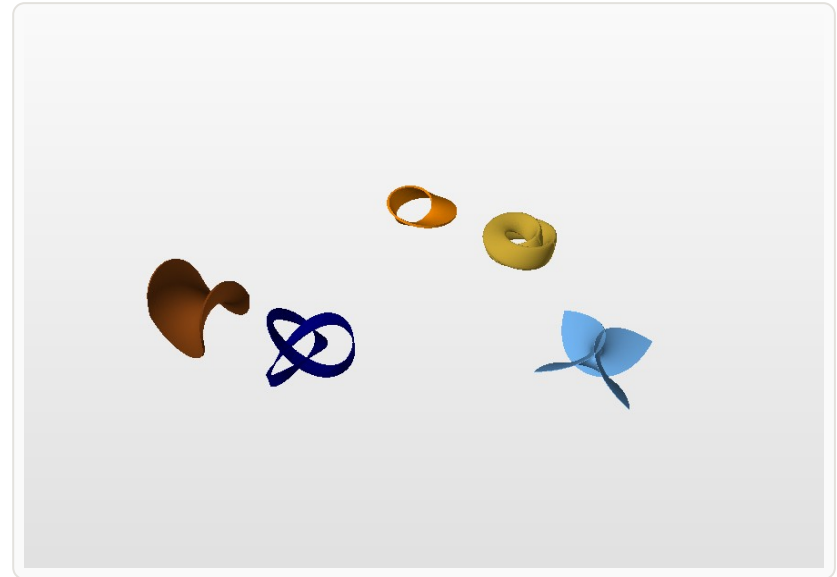


Kriauklė, sraigė, Dini paviršius, obuolys, citrina, širdis, ragas, trimitas, banguota kolona, „breather“, raibuliai, lašas.
`examples/09_surfaces_nature.py`

Grafikai ir egzotiški paviršiai



$y = f(x, z)$ · 10_height_fields.py



Miobijus, Kleinas, Boy, Romos paviršius · 07_surfaces_exotic.py

Vienpusiai paviršiai neturi „išorės“. Duokite jiems storį – ir jie tampa įprastais dvipusiais kūnais.

Kai paviršius iš vienos pusės juodas



Paprastas lakštas

Viena gera pusė, viena „negyva“.

`double_sided=True`

Kiekviena siena ir jos apversta kopija. Pigu, storio vis dar nėra.

`thickness=0.1`

Tikras kūnas su sienele: uždaras, spausdinamas, teisingas iš visų pusių.

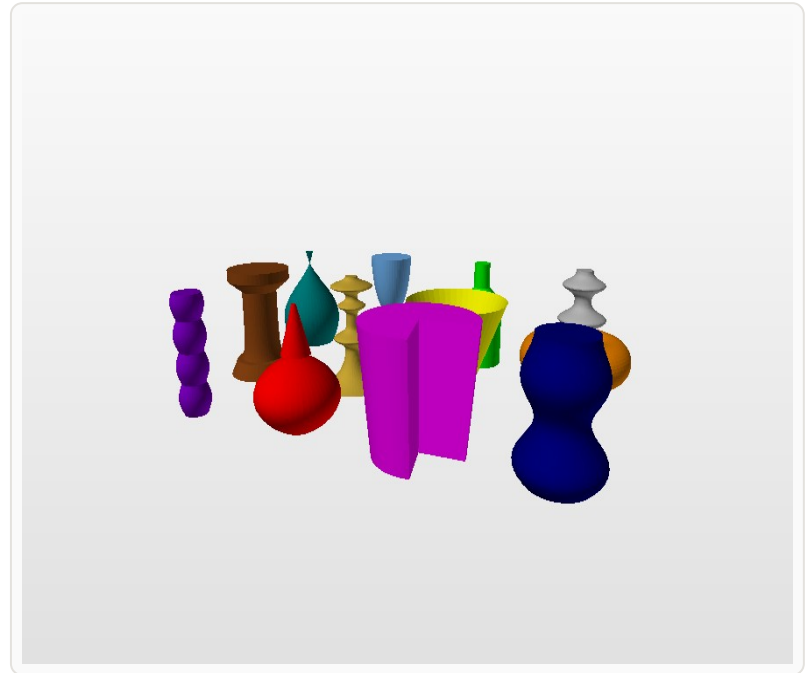
revolve() – sukinys

```
def vaza(t):  
    return [1 + 0.4*add.sin(3*t), t]  
  
add.revolve(vaza, [0, 0, 0], [0, 1, 0],  
            0, 4, 60, 40, "teal")
```

profilis(t) grąžina [spindulys, aukštis]

caps=True uždaro galus – gaunamas kūnas, tinkamas loginėms operacijoms

angle= mažesnis už pilną apsisukimą išpjauna pleištą

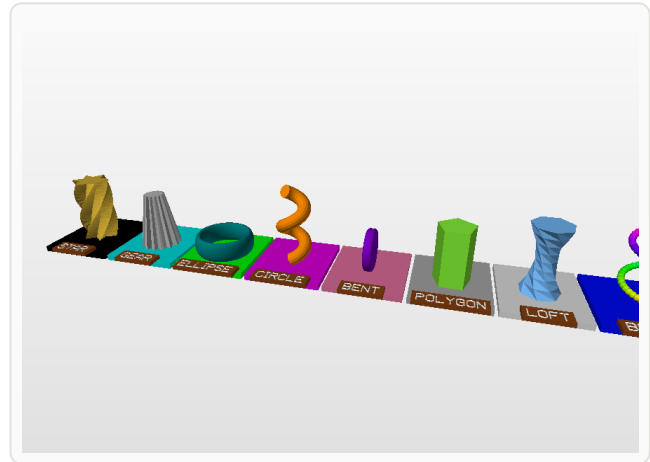


Senasis spin3D(A, B, S, ...) tebeveikia – tai tas pats sukinys be dangtelių. [examples/14_lathe_gallery.py](#)

Kopijuojamas, sukamas, tempiamas paviršius

```
add.extrude(zvaigzde, [0, 3.4, 0], "gold",  
            steps=90, twist=1.6*add.pi,  
            scale=lambda t: 1 - 0.45*t)
```

twist ir scale gali būti ir funkcijos nuo padėties kelyje – forma keičiasi eidama. profile_star, profile_gear, profile_rect ... duoda kontūrus.
examples/13_sweep_and_twist.py, 33_cross_sections.py



extrude

2D forma ištempinama į 3D; pakeliui sukama ir siaurinama.

sweep

Skerspjūvis stumiamas išilgai bet kokio 3D kelio.

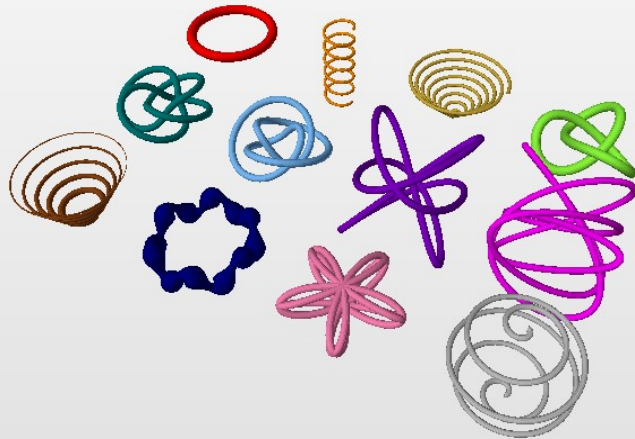
loft

Paviršius, uždengiantis skerspjūvių seką.

polyline, trace

Vamzdis per taškų sąrašą; vektorinio lauko trajektorija.

curve() – kreivės kaip vamzdžiai



```
def mazgas(t):  
    return [add.sin(t) +  
            2*add.sin(2*t),  
            -add.sin(3*t),  
            add.cos(t) -  
            2*add.cos(2*t)]
```

```
add.curve(mazgas, 0, 2*add.pi,  
          300, 18, 0.28, "lime", True)
```

Spindulys gali būti ir funkcija $r(t)$ – vamzdis storėja ir plonėja. Vamzdis pats nesisuka: išilgai kreivės nešamas minimaliai sukantis rėmas.

Transformacijos

move, place, align, ground

Pastūmimas; centrą, kampą ar apačią pastato į nurodytą tašką.

rotateX/Y/Z, rotate, aim

Sukimas apie ašį – bet kokią – arba detalės nukreipimas kryptimi.

zoom, stretch, fit

Mastelis: vienodas, atskiras kiekvienai ašiai arba pagal dydį.

mirror

Atspindys plokštumoje; sienų kryptis apsukama.

twist, bend, taper

Susukimas, sulenkimas, siaurėjimas.

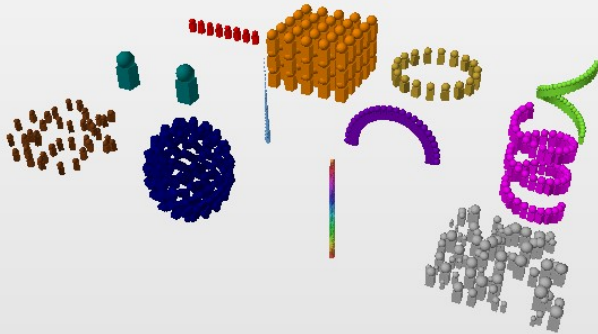
deform

Bet kokia jūsų funkcija $f(p) \rightarrow$ naujas taškas.

Visos jos grąžina naują modelį – originalas nepakinta, todėl jas galima jungti į grandinę:

```
M = add.move(add.rotateY(add.zoom(M, 0.5), 0.4), [0, 3, 0])
```

Viena detalė, daug kopijų



```
array_linear(M, step, n)
```

```
array_grid(M, steps, counts)
```

```
array_radial(M, n, axis, P)
```

```
array_mirror(M, point, normal)
```

```
scatter(M, points, seed)
```

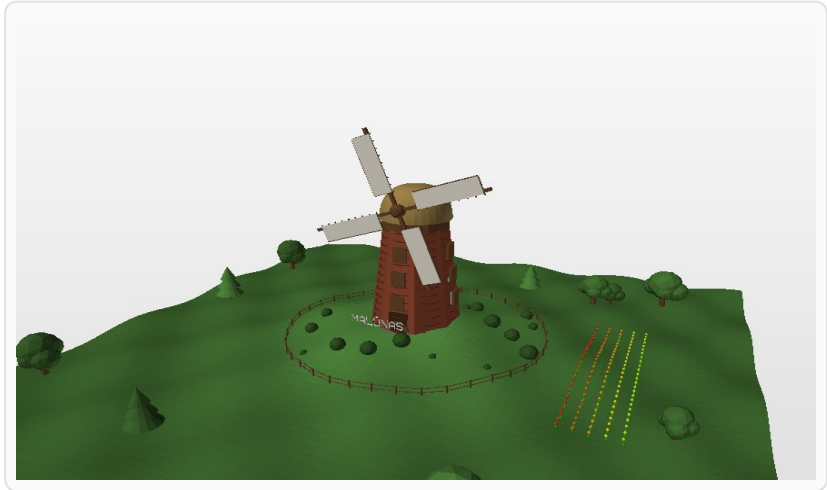
```
along(M, path, n)
```

```
spirale = add.repeat(plyta, 40,  
    lambda M, i: add.move(  
        add.rotateY(M, i * 0.3),  
        [0, i * 0.2, 0]))
```

Detalės, kurių prireikia kiekvienam modeliui



23_locomotive.py – wheel, beam, rotate_point, array_mirror



24_windmill.py – extrude, arch, stairs, tree, scatter, along

beam(A, B, w, h)

Sija tarp dviejų taškų – orientaciją apskaičiuoja pati.

roof, column, bricks, tree

Namai, šventyklos, sienos, miškai.

wheel, gear, arch, stairs

Ratai su stipiniais, sukimbantys krumpliaračiai, arkos, laiptai.

pixels, heightmap, text

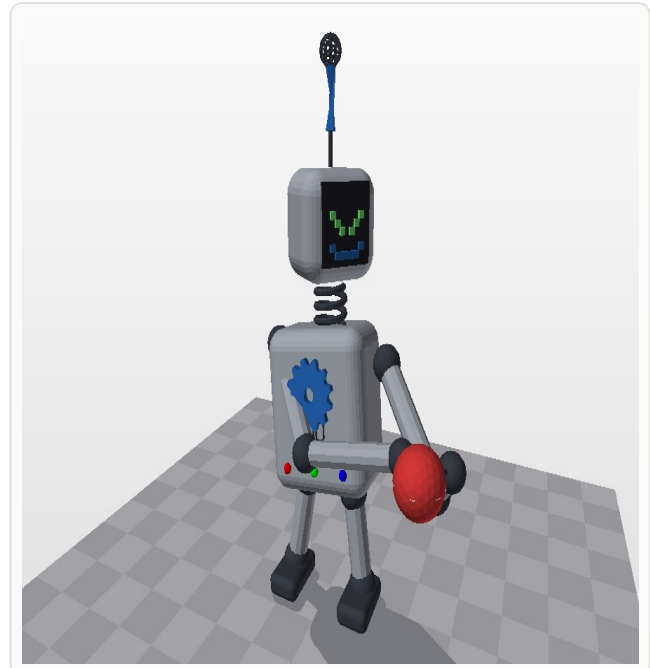
Pikselinis piešinys, kubelių reljefas, užrašai įmontuotu šriftu (ir AČĖĖĮŠŲŪŽ).

Išdėstymas: aim, scatter, along

```
# detalė aplink koordinačių pradžią, žiūri aukštyn
add.push()
add.capsule([0, 0, 0], [0, 2, 0], 0.2, 12, "silver")
arm = add.pop()

# nukreipti nuo peties į plaštaką, perkelti
arm = add.aim(arm, add.direction(shoulder, hand))
add.mesh(add.move(arm, shoulder))

# keturiasdešimt medžių ant kalvų
spots = add.random_points(40, lo, hi, seed=1,
                           height=hills)
add.mesh(add.scatter(tree, spots, seed=1,
                      scale=(0.7, 1.4)))
```



27_robot.py – kiekviena galūnė nukreipta į tašką

Kiekvieną detalę kurkite aplink koordinačių pradžią tarp push() ir pop(); tada nukreipkite, pasukite ir perkeltite į vietą.

Spalvos

```
add.box(c, 1, "red")           # vardu
add.box(c, 1, "#ff8800")      # hex
add.box(c, 1, [255, 136, 0])  # RGB

add.hsv(0.3, 0.7, 1.0)        # atspalvis
add.gradient(t, "navy", "gold") # perėjimas
add.shade("red", 0.6)          # tamsiau

M = add.color_by(M,
    lambda p: add.hsv(p[1] / 10.0))
```

Užduočiai reikia bent trijų spalvų – bet gražiau, kai jos ką nors reiškia.

color(M, spalva)

Perdažo visą modelį.

color_by(M, fn)

Spalva pagal sienos padėtį – vaivorykštės, aukščio žemėlapiai.

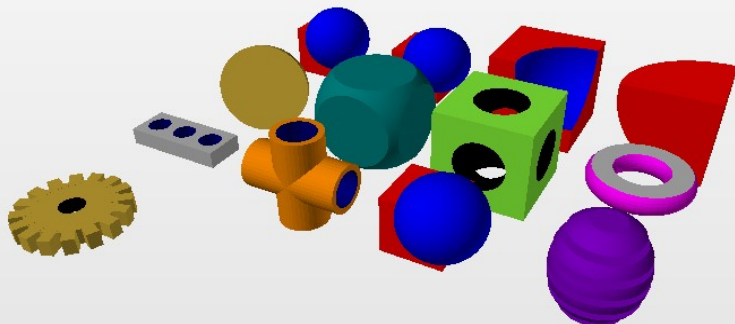
color_gradient

Perėjimas nuo vienos spalvos iki kitos išilgai ašies.

color_random

Kiekvienai sienai – sava atsitiktinė spalva.

Loginės operacijos su kūnais



union(A, B)

Sajunga: viskas, kas yra bet kuriame kūne. Paslėptos vidinės sienos dingsta.

intersect(A, B)

Sankirta: tik tai, kas bendra.

difference(A, B)

Skirtumas: A su iškirptu B. Taip daromos skylės ir įpjovos.

cut(M, taškas, normalė)

Pjūvis plokštuma – pigiau.

```
add.cuboid([0,0,0], [4,1,4], "brown")
plokste = add.layer()
add.cylinder([0,-1,0], [0,1,0], 0.6, 32, "black")
grezlas = add.layer()
add.mesh(add.difference(plokste, grezlas))
```

Kaip veikia loginės operacijos

1. Perpjaunama

Kiekviena siena perpjaunama ten, kur ją galėtų kirsti kito kūno trikampiai. Kaimynai randami per erdvinį tinklėlį, todėl toli esanti siena nepjaustoma.

2. Klausinama

Kiekvienos dalies klausinama: ar ji viduje, išorėje, ar guli ant kito kūno paviršiaus? Atsako spindulys, skaičiuojantis susikirtimus.

3. Atrenkama

Sąjunga palieka tai, kas išorėje; sankirta – kas viduje; skirtumas – A išorę ir apverstą B vidų.

Apie 600 eilučių gryno Python. Jokių bibliotekų.

6000 sienų sfera minus dėžė – apie 0,5 s. 25 000 sienų – apie 2 s.

Loginėms operacijoms reikia uždarytų kūnų. Jei rezultatas keistas – pirmiausia patikrinkite abu pradinius kūnus su `add.check()`.

Modelio taisymas ir tikrinimas

clean(M)

Suklijuoja sutampančias viršūnes, pašalina nulinio ploto ir pasikartojančias sienas, išmeta vidines sieneses ten, kur du kūnai liečiasi.

heal(M)

Užtaiso plyšelius ten, kur briauna praeina pro svetimą viršūnę.

fix_normals(M)

Nukreipia visas sienas ta pačia kryptimi, uždarame modelyje – į išorę.

triangulate(M), inside(M, p)

Visos sienos – trikampiai; ar taškas yra kūno viduje?

```
modelis = add.clean(add.layer())
add.mesh(modelis)
add.check()
```

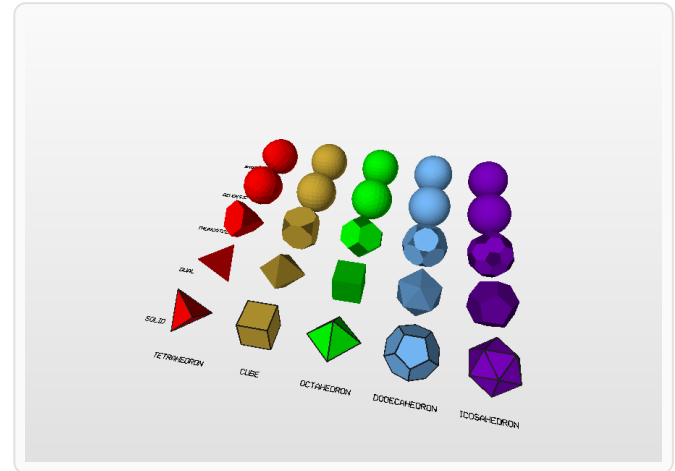
vertices	136768
OK polygons	163132
OK colours	6
OK closed surface	yes
volume	60.286

Sudėjus du besiliečiančius kubus, kiekvienas pasilieka savo sienelę toje vietoje, kur jos niekas nemato. clean() jas randa ir pašalina. [examples/31_workbench.py](#) rodo visus taisymo įrankius.

Penki taisyklingieji briaunainiai ir sfera iš trikampių

```
add.icosahedron([0, 0, 0], 2, "purple")
add.dodecahedron([5, 0, 0], 2, "gold")
P = add.polyhedron_points("icosahedron")

add.sphere([0, 5, 0], 2, 20, "sky")    # 5120 tri
add.icosphere([5, 5, 0], 2, 3, "red")  # 1280
add.quadsphere([9, 5, 0], 2, 12, "gold")
```



38_polyhedra.py – kūnai, dualieji, nupjovimai, kupolai, glotnūs

Sucentruota

Viršūnių vidurkis yra lygiai centras; kiekviena viršūnė atstumu r .

Kupolo principas

Kiekvieną trikampį į keturis, vidurio taškus – ant sferos, kartoti: 20, 80, 320, 1280 ...

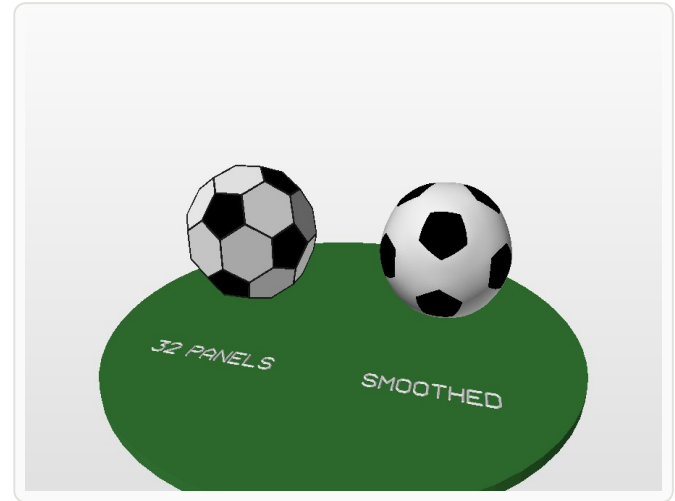
k kaip anksčiau

$\text{sphere}(c, r, k)$ lygį parenka pagal k : $k=10 \rightarrow 1280$, $k=20 \rightarrow 5120$ sienų.

Futbolo kamuolys iš ikosaedro viršūnių

```
ico = add.make(add.icosahedron, [0, 0, 0], 3)
print(add.neighbors(ico, 0))  # 5, in order
print(add.valence(ico, 0))

ball = add.truncate(ico, 1 / 3.0)  # corners off
ball = add.color_by_sides(ball,
                             {5: "black", 6:
                              "white"})
add.mesh(add.smooth(ball, 16))      # rounded
```



37_football.py

Paklauskite tinklo

neighbors, valence,
mean_neighbor_distance,
edges, vertex_normal,
face_center, boundary_loops.

Pakeiskite vieną kampą

set_vertex(M, i, [x, None, z])
perkelia viršūnę i ir palieka
sienas – tada smooth().

Perkurkite

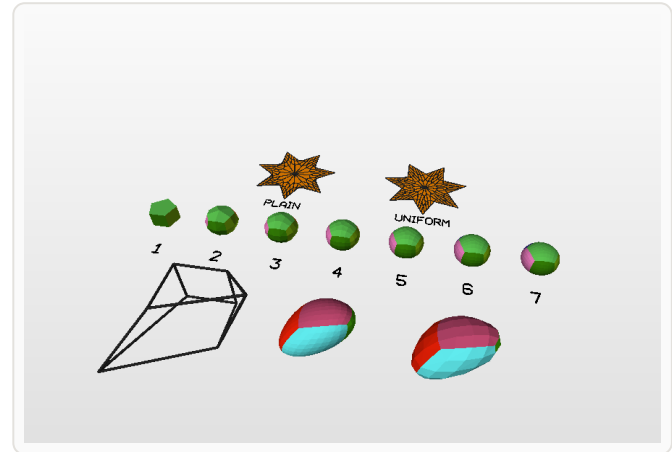
dual, truncate, refine + spherify,
inflate – nauji kūnai iš senų.

SMOOTH()

Glotnūs paviršiai: apibendrintas Catmull–Clark

```
add.box([0, 0, 0], 2, "gold")
block = add.layer()
i = add.nearest_vertex(block, [1, 1, 1])
block = add.set_vertex(block, i, [2.5, 2.5, None])

add.mesh(add.smooth(block, 8))    # 8 per edge
add.mesh(add.catmull_clark(block, 3))
```



39_smooth_shapes.py – n = 1 ... 7 ant vienos prizmės

Bet koks n

n langelių ant kiekvienos kontrolinės briaunos, n = 1, 2, 3, 4, 5 ... – klasikinis dalijimas duoda tik 2, 4, 8.

Tiksliai ant ribinio paviršiaus

Kiekviena nauja viršūnė skaičiuojama ant ribinio paviršiaus; prie ypatingųjų kampų tinklas perparametrizuojamas, kad liktų tolygus.

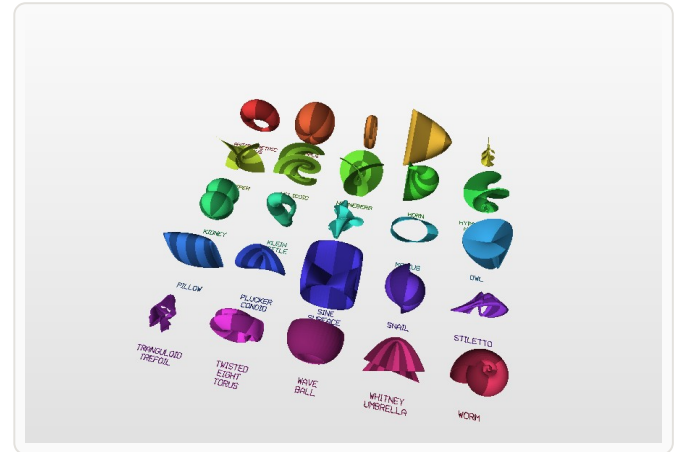
Straipsnis

Sabaliauskas, Uniform n-grids on Catmull–Clark limit surfaces of arbitrary polygon meshes (2026); perkelta eilutė po eilutės į gryną Python.

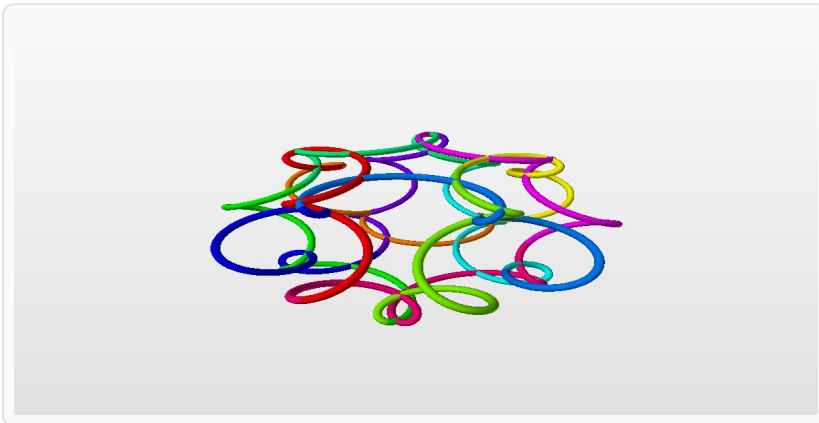
Dvidešimt penki paviršiai pagal vardą

```
print(add.surface_names())
add.surface("klein_bottle", [0, 0, 0], 4, 120)
add.surface("dini", [6, 0, 0], 4,
              color=lambda u, v: add.hsv(u / 12))
add.surface("pillow", [-6, 0, 0], 3, a=0.9)

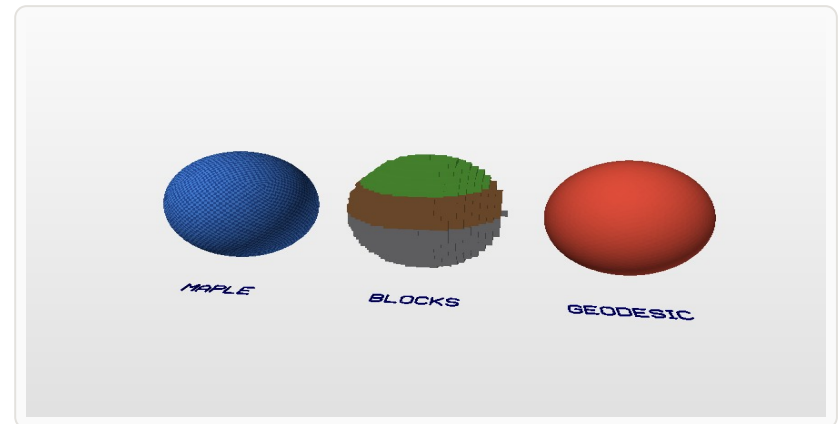
owl = add.surface_function("owl")    # bare f(u, v)
```



34_surface_zoo.py



35_knot_curve.py – mazgas iš besisukančių apskritimų, vienas
curve() kvietimas

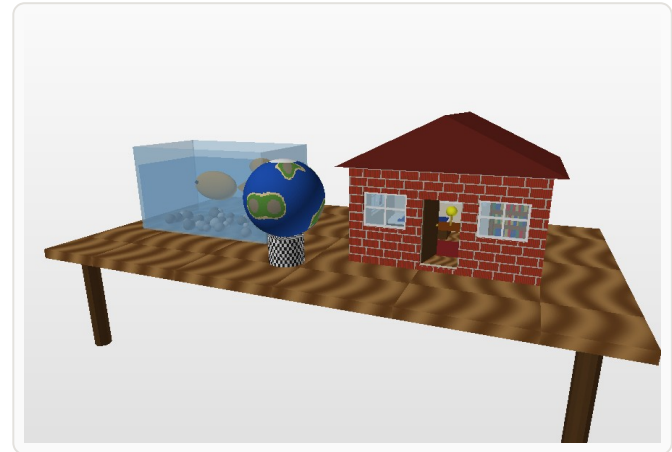


36_minecraft_sphere.py – Maple, kubeliai, geodezinė

Stiklas, paveikslėliai ir Sketchfab ribos

```
glass = add.transparent("sky", 0.35)      # 0..1
wall = add.make(add.cuboid, [0, 1.5, 0], [6, 3, 0.3])
hole = add.make(add.cuboid, [0, 1.6, 0], [2, 1.2, 1])
wall = add.difference(wall, hole) # pirma anga,
add.mesh(add.texture(wall, "bricks.png")) # tada
# stiklas angoje
add.cuboid([0, 1.6, 0], [2, 1.2, 0.03], glass)

add.check()      # <= 50 MB, <= 50 colours
add.save("house.obj", colors=50)  # .mtl
```



45_glass_and_textures.py – pro stiklą matyti kambarys

Neprivaloma

Be transparent/texture viskas lygiai kaip anksčiau; .off failuose lieka paprastos spalvos.

Ribos

Sketchfab: 100 MB nemokamai, medžiagos virš 100 suliejamos.
Kursas: 50 MB, 50 spalvų.
Padeda limit_colors() ir obj_size().

Paveikslėliai iš kodo

write_png("bricks.png", rows)
įrašo patį paskaičiuotą paveikslėlį – failų nereikia.

Pilis: modelis, didesnis už atmintį



be tekstūrų, kiekvienas akmuo – daugiakampis: apie 1 GB .off ir .obj, kurį 7-Zip suglaudina į ~100 MB Sketchfab'ui



sosto menė ir puota



drakonas ant lobio

```
out = add.stream("castle.off", precision=4)  # parts go to disk at once,
wall_of_bricks(); out.add()                 # tidied (no overlaps), written, cleared
out.close()                                # fills in the OFF header (.mtl for .obj)
```

Pilis Sketchfab'e – pasukiokite ją 3D: skfb.ly/pOnRS

Modelio išsaugojimas ir viešinimas

```
add.save("modelis.off")    # kurso formatas  
add.save("modelis.obj")    # + .mtl spalvoms  
add.save("modelis.ply")  
add.save("modelis.stl")    # 3D spausdinimui
```

MeshLab konvertavimo žingsnis nebereikalingas – add.py OBJ ir MTL parašo pats.

Sketchfab

Sudėkite .obj ir .mtl į vieną archyvą ir įkelkite – modelį galės pasukioti bet kas naršyklėje, kaip pilį:

skfb.ly/pOnRS

3D spausdinimas

clean(..., normals=True), tada save("m.stl"). Paviršius turi būti uždaras.

Peržiūra be MeshLab

python3 tools/preview.py modelis.off sukuria PNG.
Irgi be jokių bibliotekų.

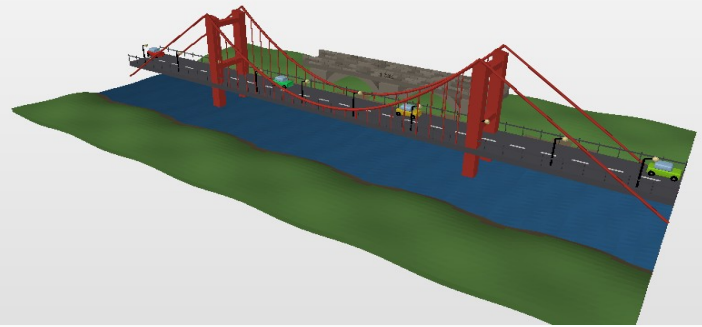
Įkėlimas atgal

load("raide.off") grąžina modelį, kurį galima stumdyti ir dažyti.

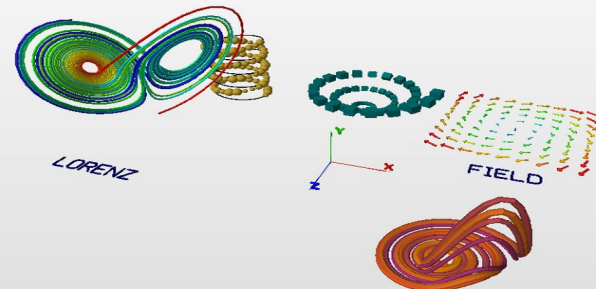
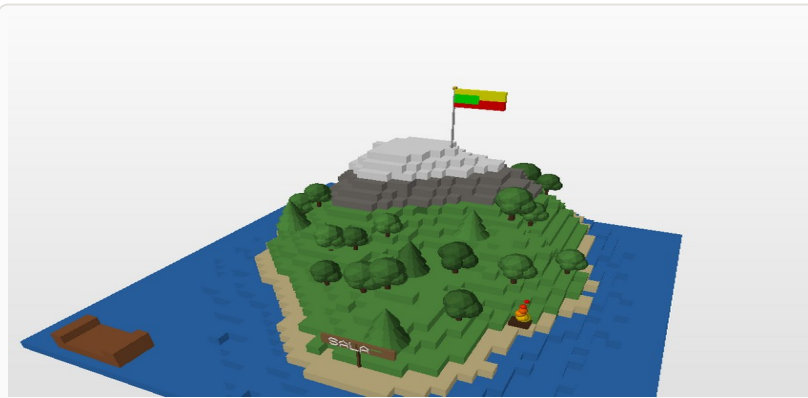
Pilni modeliai



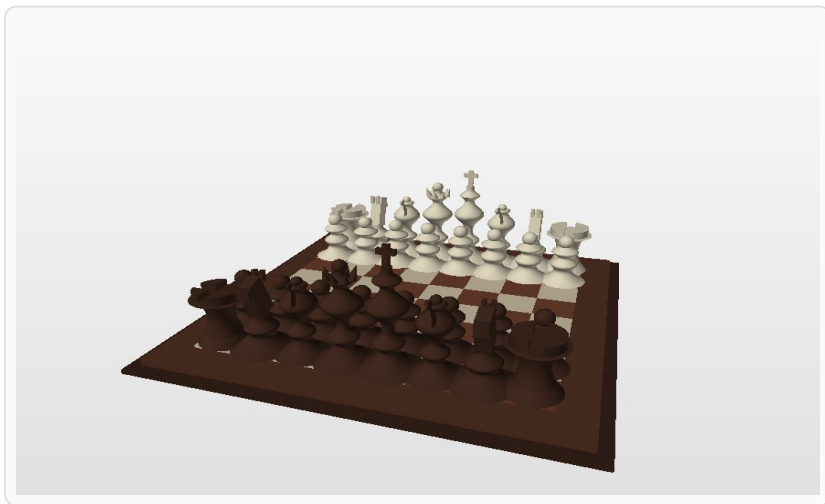
25_temple.py – kolonos ant apskritimo, languotos grindys



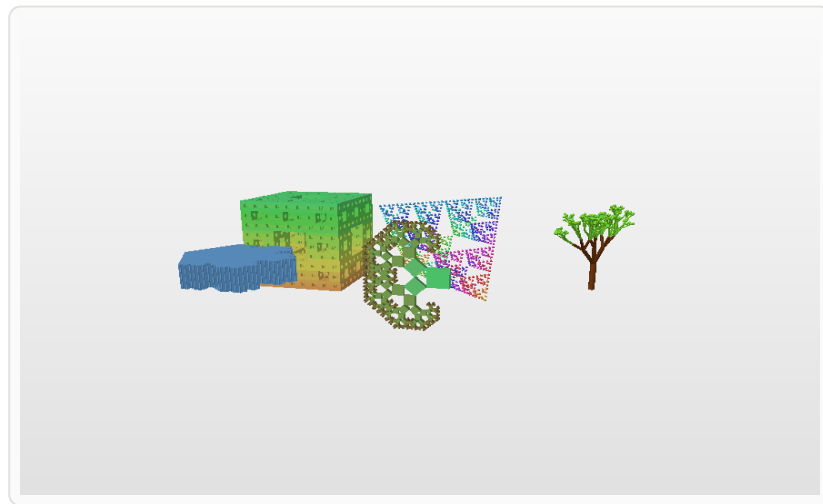
29_bridge.py – sijos, lynai, pakabos, automobiliai kelyje



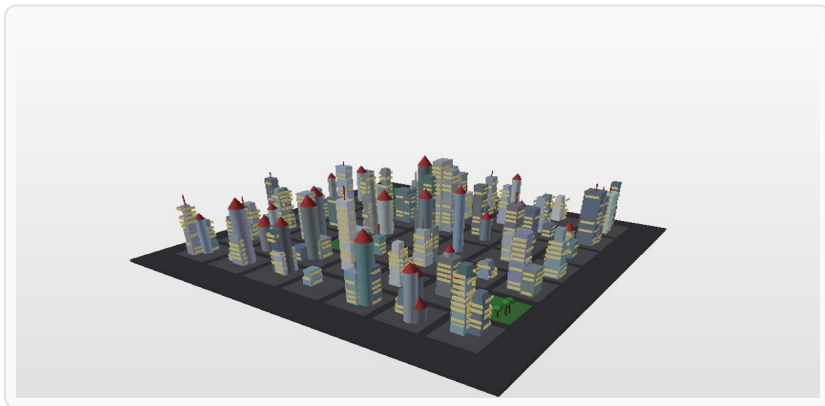
Didesnės studijos



18_chess_set.py – sukinys, sluoksniai, loginės operacijos



17_fractals.py – Mengerio kempinė, Sierpinskis, medis



Nuo 1.2 prie 2.0

add.py 1.2	add.py 2.0	kas tai
<code>cube2(c, e, b, RGB)</code>	frame	kubo briaunų karkasas
<code>cylinder2(A, B, r, k, RGB)</code>	tube	cilindras be dangtelių
<code>cylinder3(A, B, r, k, RGB)</code>	cup	uždarytas iš vieno galo
<code>cone2(A, B, r, k, RGB)</code>	cone_open	tik šoninis paviršius
<code>rectangle3D(c, e, RGB)</code>	cuboid	stačiakampis gretasienis
<code>newface(A, RGB)</code>	polygon	viena plokščia siena
<code>spin3D(A, B, S, ...)</code>	revolve	sukinys
<code>off(failas)</code>	save(failas)	.off, .obj, .ply arba .stl

Visi seni vardai tebeveikia. Dvylika 1.2 versijai rašytų modelių guli tests/legacy/ aplanke, ir testai tikrina, kad jie duotų lygiai tiek pat sienų kaip anksčiau (pasikeitė tik sphere: dabar geodezinė; senoji – quadsphere).
examples/32_old_names.py – ištisas modelis 1.2 žodynu.

Kur viską rasti

add.py ir add.hpp, pavyzdžiai, testai ir dokumentacija

github.com/.../add3d

Dokumentacija angliškai ir lietuviškai

...github.io/add3d

Klausimai

akataxis@gmail.com

Ačiū už dėmesį.

